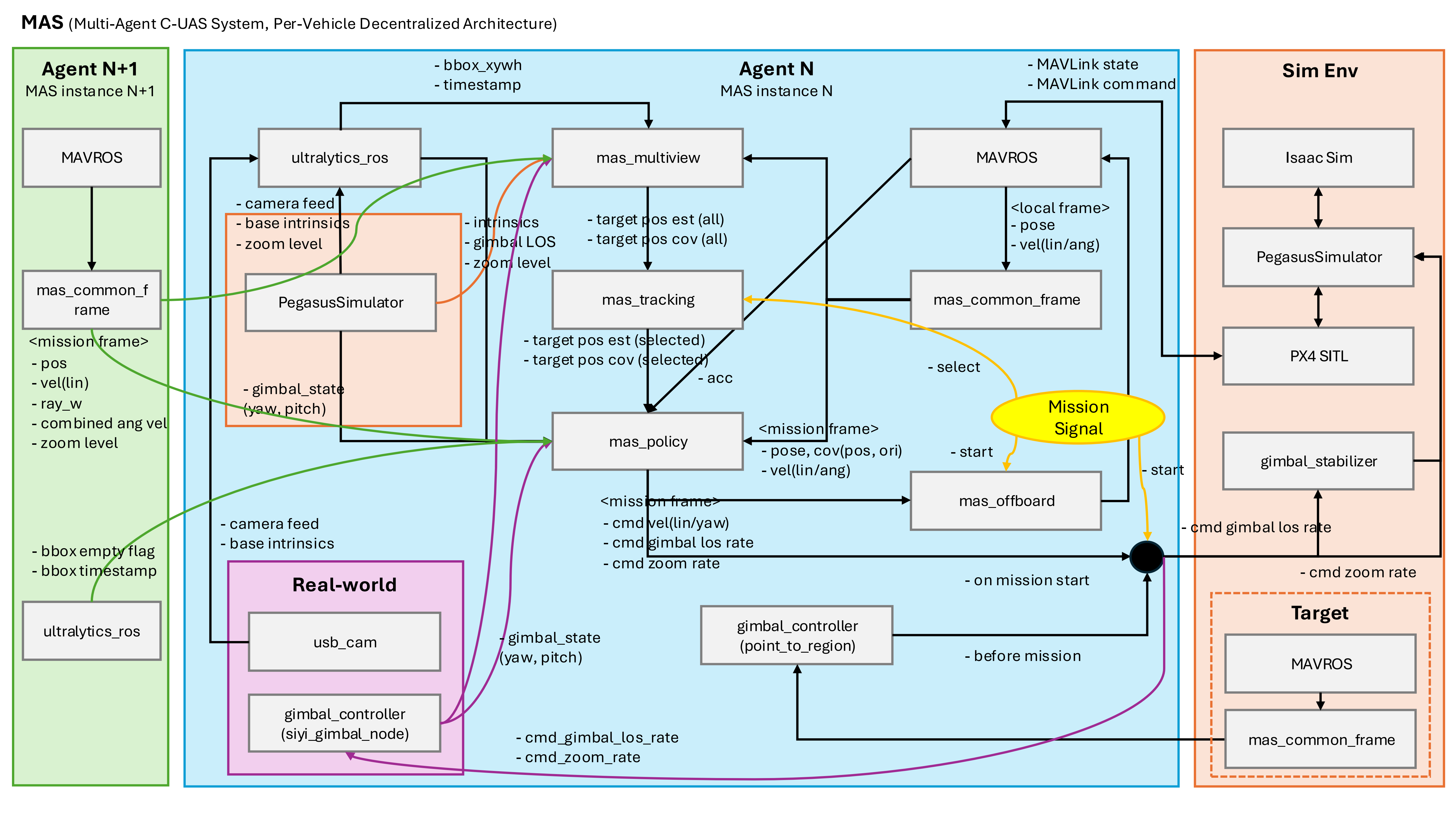
MAS (Multi-Agent C-UAS System, Per-Vehicle Decentralized Architecture)
Agent N+1
MAS instance N+1
MAVROS
mas_common_frame
<mission frame>
- pos
- vel(lin)
- ray_w
- combined ang vel
- zoom level
- bbox empty flag
- bbox timestamp
ultralytics_ros
Agent N
MAS instance N
- bbox_xywh
- timestamp
ultralytics_ros
mas_multiview
- camera feed
- base intrinsics
- zoom level
PegasusSimulator
- intrinsics
- gimbal LOS
- zoom level
- target pos est (all)
- target pos cov (all)
mas_tracking
- target pos est (selected)
- target pos cov (selected)
- acc
- gimbal_state
(yaw, pitch)
mas_policy
<mission frame>
- pose, cov(pos, ori)
- vel(lin/ang)
<mission frame>
- cmd vel(lin/yaw)
- cmd gimbal los rate
- cmd zoom rate
- camera feed
- base intrinsics
Real-world
usb_cam
gimbal_controller
(siyi_gimbal_node)
- gimbal_state
(yaw, pitch)
- cmd_gimbal_los_rate
- cmd_zoom_rate
gimbal_controller
(point_to_region)
- MAVLink state
- MAVLink command
MAVROS
<local frame>
- pose
- vel(lin/ang)
mas_common_frame
- select
Mission
Signal
- start
- start
mas_offboard
- on mission start
- before mission
Sim Env
Isaac Sim
PegasusSimulator
PX4 SITL
gimbal_stabilizer
- cmd gimbal los rate
- cmd zoom rate
Target
MAVROS
mas_common_frame

MAS (Multi-Agent System) Semantic-level Architecture

Objective
Define system components and interfaces in `MAS` in semantic-level.
- What kind of system components are needed?
- What kind of information is exchanged?

The Semantic-level System Architecture
Groups
- Agent N, Agent N+1
- Sim Env
- Real-world

Components
- Simulator, ROS2 nodes, drivers

Interface
- Goal: make Sim Env identical to Real-world
- Note: may differ from the current ROS2 node implementation

Decisions to be made

Do we need a dedicated mission planner?(`mas_mission`)
`mas_offboard` already gates cmd_vel via its state machine (HOVER→POLICY). The parallel problem for gimbal/zoom doesn't have an equivalent gate. A mas_mission node could:

- Own a simple state machine: `IDLE → TRACKING → MISSION`
- In `TRACKING`: enable `point_to_region_node`, disable policy gimbal commands
- In `MISSION`: disable `point_to_region_node`, enable policy commands
- Expose a service for the operator to trigger transitions
- Publish a `mission_state` topic that other nodes can check

This keeps it small — it's a state publisher + service server, not a full planner.